



Manual de Usuario

Tienda en Línea Mystic Box Versión: 1.0

Desarrolladores
Betsi Quiros López
Estefani Madrigal Carvajal

Universidad Latina de Costa Rica
Proyecto de Ingeniería en sistemas

Curso
Programación V

Profesor Lic. Osvaldo Granados Venegas

2026

Contenido

1. Introducción	5
2. Objetivos del Documento	5
2.1 Objetivo general	5
2.2 Objetivos específicos	5
3. Descripción General del Sistema	5
4.1 Vista general	6
4.2 Backend — capas del proyecto .NET	6
4.2.1 Patrón Repositorio genérico + Unidad de Trabajo	7
4.3 Frontend — aplicación Ionic + Angular	7
4.4 Comunicación Frontend – Backend	7
5.1 Backend	8
5.2 Frontend	8
5.3 Herramientas de desarrollo	8
6. Modelo de Base de Datos	9
6.1 Relaciones principales	9
6.2 Diccionario de datos	9
Usuarios	9
Roles	10
Categorías	10
MysticBox	10
Carrito	10
DetalleCarrito	11
Pedidos	11
DetallePedido	11
Entregas	12
Facturas	12
Calificaciones	12
Cupones	12
MetodosPago	13
Personalizaciones	13
WhiteList	13
7.1 Catálogo de endpoints	14
AuthController (api/Auth)	14
UsuarioController (api/Usuario)	14
RolController (api/Rol)	14
CategoriasController (api/categorias)	15

MysticBoxController (api/mysticbox)	15
PersonalizacionesController (api/Personalizaciones)	15
CarritoController (api/Carrito)	15
DetalleCarritoController (api/DetalleCarrito)	16
CuponesController (api/cupones)	16
PedidoController (api/Pedido)	16
DetallePedidoController (api/DetallePedido)	16
EntregaController (api/Entrega)	16
FacturasController (api/facturas)	17
CalificacionController (api/Calificacion)	17
WhiteListController (api/whiteList)	17
DashboardController (api/Dashboard)	17
EstadisticasController (api/Estadisticas)	17
7.2 Autenticación	18
8.1 Estructura de rutas	18
8.2 Guardas de acceso (Guards)	19
8.3 Servicios (capa de acceso a la API)	20
8.4 Manejo de sesión en el cliente	20
8.5 Empaquetado como aplicación Android (APK)	20
8.5.1 Pasos para generar el APK	22
9.1 Gestión de usuarios y autenticación	23
9.2 Catálogo y categorías	23
9.3 Personalización de cajas	24
9.4 Carrito de compras	24
9.5 Cupones y promociones	24
9.6 Pedidos, facturación y métodos de pago	25
9.7 Entregas	25
9.8 Calificaciones	25
9.9 Lista blanca (WhiteList)	25
9.10 Panel administrativo: dashboard, reportes y estadísticas	26
10. Instalación y Configuración del Entorno	27
10.1 Requisitos previos	27
10.2 Configuración de la base de datos	27
10.3 Configuración y ejecución del backend	27
10.4 Configuración y ejecución del frontend	27
10.5 Generación del APK (aplicación Android)	28
10.6 Usuario administrador inicial	28

11.1 Aspectos correctamente implementados28

1. Introducción

Este manual técnico documenta el diseño, la arquitectura y el funcionamiento del sistema Mystic Box, una plataforma de comercio electrónico desarrollada para la venta de cajas temáticas ("boxes") personalizadas por categorías, como anime, belleza, gaming, oficina, autocuidado e infantil. El sistema permite a los clientes explorar el catálogo, personalizar sus cajas, gestionar un carrito de compras, generar pedidos, dar seguimiento a entregas y calificar su experiencia, mientras que el personal administrativo cuenta con un panel para gestionar usuarios, catálogo, promociones, pedidos y reportes estadísticos.

El proyecto está construido bajo una arquitectura cliente-servidor separada en dos grandes componentes: un backend expuesto como API REST desarrollado en ASP.NET Core (.NET 8) con Entity Framework Core sobre SQL Server, y un frontend híbrido desarrollado con Ionic y Angular, capaz de ejecutarse como aplicación web y como aplicación móvil mediante Capacitor.

Este documento está dirigido a desarrolladores, evaluadores del curso y cualquier persona que necesite comprender la estructura interna del sistema, instalarlo en un entorno de desarrollo o darle mantenimiento.

2. Objetivos del Documento

2.1 Objetivo general

Describir de forma clara y estructurada la arquitectura, los componentes y el funcionamiento técnico del sistema Mystic Box, de manera que sirva como referencia para su instalación, comprensión, mantenimiento y evaluación académica.

2.2 Objetivos específicos

- Describir la arquitectura general del sistema y la responsabilidad de cada capa.
- Detallar el modelo de base de datos, sus entidades y relaciones.
- Catalogar los endpoints expuestos por la API REST del backend.
- Describir la estructura del frontend, sus rutas, guardas de acceso y servicios.
- Explicar los módulos funcionales que componen el sistema desde la perspectiva del usuario.
- Indicar los pasos necesarios para instalar y ejecutar el sistema en un entorno local.

3. Descripción General del Sistema

Mystic Box es una tienda en línea especializada en la venta de cajas sorpresa temáticas ("MysticBox"), agrupadas en categorías como Anime, Belleza, Gaming, Oficina, Autocuidado e Infantil. Cada caja puede adquirirse en distintas presentaciones (básica, deluxe, premium) y el cliente puede personalizarla indicando tamaño, preferencias, exclusiones y un mensaje personalizado.

El sistema contempla dos perfiles de usuario principales:

- Cliente: se registra, inicia sesión, navega el catálogo por categorías, arma su carrito, aplica cupones de descuento, genera pedidos, elige método de pago, da seguimiento a la entrega y califica el pedido recibido.

- **Administrador:** gestiona usuarios, roles, categorías, catálogo de cajas, cupones y promociones, supervisa pedidos y entregas, administra una lista blanca (whitelist) de correos autorizados, y consulta un panel de estadísticas y reportes (dashboard, satisfacción del cliente, reportes generales).

La aplicación cliente se distribuye como una aplicación híbrida construida con Ionic Framework sobre Angular, lo que permite ejecutarla tanto en navegador web como empaquetada para dispositivos móviles (Android/iOS) mediante Capacitor. El backend expone toda la lógica de negocio como una API REST documentada con Swagger.

4. Arquitectura del Sistema

El sistema sigue una arquitectura en capas de tipo N-Capas en el backend, combinada con un patrón cliente-servidor frente al frontend. A continuación se describe cada componente.

4.1 Vista general

El flujo general de una solicitud es el siguiente: la aplicación Ionic/Angular realiza una petición HTTP hacia la API (por ejemplo, `https://localhost:7216/api/...`); el controlador correspondiente en MysticBox.Api recibe la solicitud y delega en la capa de Lógica de Negocio; esta a su vez utiliza la Unidad de Trabajo (Unit of Work) para acceder a los repositorios genéricos de la capa de Acceso a Datos, los cuales usan Entity Framework Core para interactuar con la base de datos SQL Server; la respuesta viaja de regreso en formato JSON hasta el frontend.

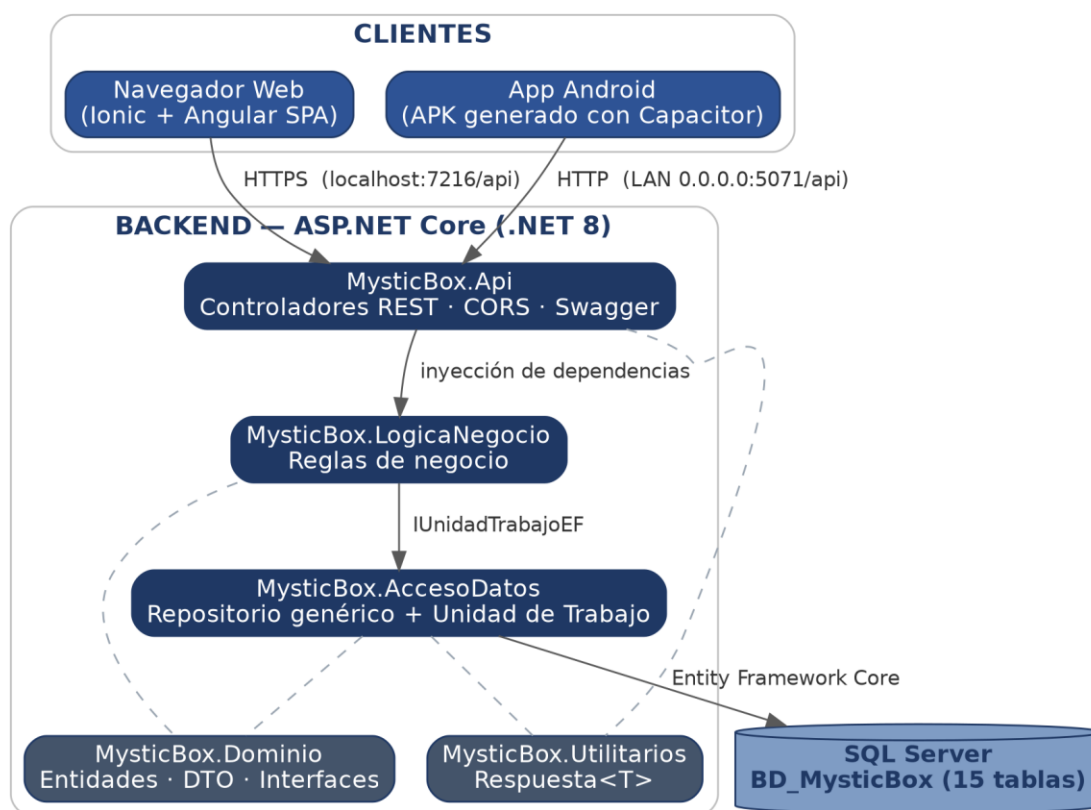


Figura 1. Arquitectura general del sistema Mystic Box.

4.2 Backend — capas del proyecto .NET

El backend está organizado en cinco proyectos (.csproj) dentro de una misma solución (MysticBox.Api.slnx), siguiendo el principio de separación de responsabilidades:

Proyecto	Responsabilidad	Contenido principal
MysticBox.Api	Capa de presentación / API REST	Controladores REST, configuración de arranque (Program.cs), CORS, Swagger, appsettings
MysticBox.Dominio	Entidades y contratos	Entidades EF (Entidades/), DTOs (DTO/), interfaces de acceso a datos (InterfacesAD/) e interfaces de lógica de negocio (InterfacesLN/)
MysticBox.LogicaNegocio	Reglas de negocio	Implementaciones de los servicios de negocio (Implementaciones/*LN.cs) inyectados en los controladores
MysticBox.AccesoDatos	Persistencia	DbContext (MysticBoxContext), repositorio genérico y Unidad de Trabajo (UnidadTrabajoEF)
MysticBox.Utilitarios	Utilidades transversales	Clase genérica Respuesta<T> usada como envoltorio estándar de resultados

4.2.1 Patrón Repositorio genérico + Unidad de Trabajo

El acceso a datos se implementa mediante un Repositorio genérico (RepositorioAD<TEntity>) que provee operaciones CRUD estándar (Insertar, Modificar, Eliminar, Obtener) reutilizables para cualquier entidad del dominio. La clase UnidadTrabajoEF centraliza el acceso a cada repositorio (TUsuario, TRol, TCarrito, TPedido, TMysticBox, TCupon, TFactura, TWhiteList, TPersonalizacion, etc.) y coordina las transacciones contra un único DbContext (MysticBoxContext), evitando así múltiples instancias de contexto por solicitud.

Todas las operaciones devuelven una instancia de la clase genérica Respuesta<T> (definida en MysticBox.Utilitarios), que estandariza el resultado de cada operación con las propiedades Exitoso, Mensaje y Datos, facilitando el manejo uniforme de errores y éxitos en toda la aplicación.

4.3 Frontend — aplicación Ionic + Angular

El cliente (carpeta MysticBoxApp) es una aplicación Ionic 8 sobre Angular 20, construida con componentes standalone (loadComponent) y enrutamiento perezoso (lazy loading) por página. Está preparada para compilarse como aplicación móvil nativa mediante Capacitor (Android/iOS) además de ejecutarse como aplicación web.

La estructura principal del frontend es:

- src/app/pages/ — una carpeta por cada pantalla del sistema (login, home, carrito, admin-dashboard, etc.), cada una con su archivo .page.ts, plantilla .page.html y estilos .page.scss.
- src/app/services/ — servicios inyectables (HttpClient) que encapsulan las llamadas a cada controlador de la API (uno por entidad: auth, carrito, pedido, categorias, mysticbox, cupon, factura, entrega, calificacion, personalizacion, whitelist, dashboard, estadisticas, usuario).
- src/app/guards/ — guardas de ruta (authGuard, adminGuard, clienteGuard) que controlan el acceso según el estado de sesión y el rol del usuario.
- src/environments/ — configuración de entorno (URL base de la API) para desarrollo y producción.

4.4 Comunicación Frontend – Backend

La comunicación se realiza mediante peticiones HTTP/JSON usando HttpClient de Angular contra la URL configurada en environment.apiUrl. En el flujo de desarrollo web esta URL apunta a https://localhost:7216/api,

con el backend restringiendo CORS mediante una política nombrada "PermitirIonic" que autoriza explícitamente los orígenes del servidor de desarrollo de Ionic (puertos 8100, 8101 y 8102).

Para el empaquetado como aplicación Android, la API deja de ser accesible en localhost desde el dispositivo/emulador, por lo que el proyecto reconfigura ambos extremos para trabajar sobre la red local (LAN): el backend pasa a escuchar en la dirección 0.0.0.0:5071 (en vez de únicamente localhost) y el frontend apunta su apiUrl a la IP LAN del equipo que ejecuta la API (por ejemplo, <http://192.168.18.153:5071/api>). Esta variante se documenta con más detalle en la sección 8.5 y en la sección de instalación.

5. Tecnologías y Dependencias

5.1 Backend

Componente	Tecnología	Versión / detalle
Framework	.NET / ASP.NET Core	net8.0
ORM	Entity Framework Core	8.0.11 (SqlServer, Tools, Design)
Base de datos	Microsoft SQL Server	SQL Server Express (motor local)
Documentación de API	Swashbuckle (Swagger)	6.6.2
Seguridad de contraseñas	ASP.NET Core Identity — PasswordHasher<T>	Hash con salt, sin texto plano

5.2 Frontend

Componente	Tecnología	Versión
Framework SPA	Angular	20.3.25
Framework UI híbrido	Ionic Framework	^8.0.0
Empaquetado nativo	Capacitor (App, Haptics, Keyboard, StatusBar, Android)	^8.x (incluye @capacitor/android 8.4.1)
Reactividad	RxJS	~7.8.0
Lenguaje	TypeScript	según tsconfig del proyecto
Testing	Karma + Jasmine	config. en karma.conf.js

5.3 Herramientas de desarrollo

- Visual Studio / Visual Studio Code — edición y depuración del backend y frontend.
- Node.js y npm — gestión de dependencias y ejecución del servidor de desarrollo Ionic (ng serve / ionic serve).
- SQL Server Management Studio (SSMS) — administración de la base de datos BD_MysticBox.
- Git — control de versiones (el proyecto incluye historial de ramas por integrante, p. ej. feature/backend-Betsi).
- Swagger UI — pruebas manuales de los endpoints en entorno de desarrollo.

6. Modelo de Base de Datos

La base de datos BD_MysticBox se implementa en Microsoft SQL Server y está compuesta por 15 tablas relacionadas mediante claves foráneas. El script de creación se encuentra en Database/BD_MysticBox.sql. A continuación se describe el modelo relacional.

6.1 Relaciones principales

- Usuarios (N) → Roles (1): cada usuario pertenece a un rol (Administrador / Cliente).
- Carrito (N) → Usuarios (1); DetalleCarrito (N) → Carrito (1), MysticBox (1) y Personalizaciones (0..1).
- Pedidos (N) → Usuarios (1), Cupones (0..1) y MetodosPago (1).
- DetallePedido (N) → Pedidos (1), MysticBox (1) y Personalizaciones (0..1).
- Entregas (N) → Pedidos (1); Facturas (N) → Pedidos (1); Calificaciones (N) → Pedidos (1).
- MysticBox (N) → Categorías (1); Personalizaciones (N) → Usuarios (1) y MysticBox (1).
- WhiteList: lista independiente de correos autorizados para registro/control administrativo.

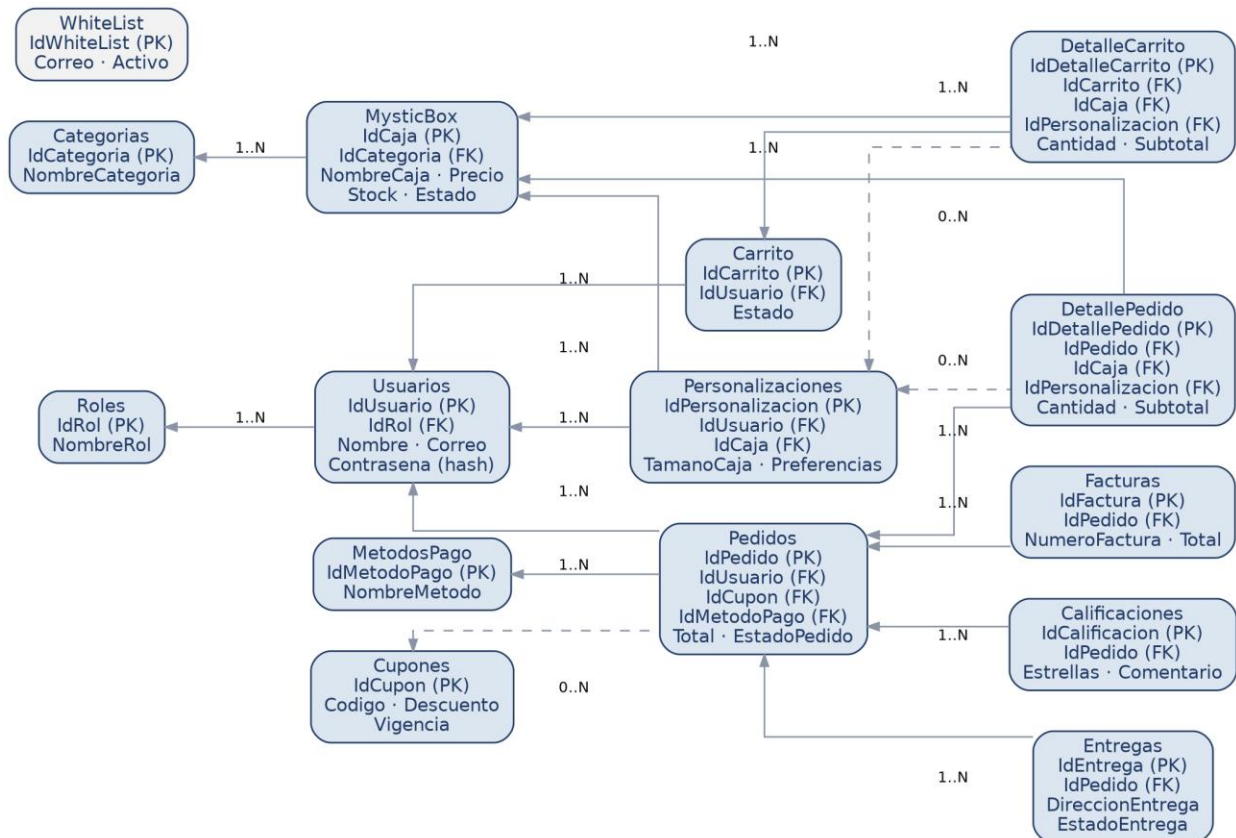


Figura 2. Diagrama entidad-relación de la base de datos BD_MysticBox.

6.2 Diccionario de datos

Usuarios

Campo	Tipo	Observación
IdUsuario	int (PK, identity)	Autonumérico

Campo	Tipo	Observación
IdRol	int (FK → Roles)	Rol asociado
Nombre	varchar(100)	Obligatorio
Correo	varchar(100)	Único, obligatorio
Telefono	varchar(20)	Opcional
Direccion	varchar(200)	Opcional
Contrasena	varchar(255)	Hash de la contraseña
FechaRegistro	datetime	Opcional
Estado	bit	Activo / inactivo

Roles

Campo	Tipo	Observación
IdRol	int (PK, identity)	Autonumérico
NombreRol	varchar(50)	Administrador, Cliente

Categorías

Campo	Tipo	Observación
IdCategoria	int (PK, identity)	Autonumérico
NombreCategoria	varchar(100)	Anime, Belleza, Gaming, etc.
Descripcion	varchar(255)	Opcional

MysticBox

Campo	Tipo	Observación
IdCaja	int (PK, identity)	Autonumérico
IdCategoria	int (FK → Categorías)	Categoría de la caja
NombreCaja	varchar(100)	Obligatorio
Descripcion	varchar(300)	Opcional
Precio	decimal(10,2)	Obligatorio
Imagen	varchar(255)	Ruta/URL de imagen
Stock	int	Unidades disponibles
Estado	bit	Activo / inactivo

Carrito

Campo	Tipo	Observación
IdCarrito	int (PK, identity)	Autonumérico
IdUsuario	int (FK → Usuarios)	Propietario del carrito

Campo	Tipo	Observación
FechaCreacion	datetime	Opcional
Estado	varchar(50)	Ej. activo, procesado

DetalleCarrito

Campo	Tipo	Observación
IdDetalleCarrito	int (PK, identity)	Autonumérico
IdCarrito	int (FK → Carrito)	Carrito asociado
IdCaja	int (FK → MysticBox)	Caja seleccionada
IdPersonalizacion	int (FK → Personalizaciones, nulo)	Personalización opcional
Cantidad	int	Obligatorio
PrecioUnitario	decimal(10,2)	Obligatorio
Subtotal	decimal (columna calculada)	Cantidad × PrecioUnitario

Pedidos

Campo	Tipo	Observación
IdPedido	int (PK, identity)	Autonumérico
IdUsuario	int (FK → Usuarios)	Cliente que ordena
IdCupon	int (FK → Cupones, nulo)	Cupón aplicado
IdMetodoPago	int (FK → MetodosPago)	Método usado
FechaPedido	datetime	Opcional
Subtotal / Descuento / Total	decimal(10,2)	Montos del pedido
EstadoPedido	varchar(50)	Ej. pendiente, enviado, entregado

DetallePedido

Campo	Tipo	Observación
IdDetallePedido	int (PK, identity)	Autonumérico
IdPedido	int (FK → Pedidos)	Pedido asociado
IdCaja	int (FK → MysticBox)	Caja comprada
IdPersonalizacion	int (FK → Personalizaciones, nulo)	Personalización opcional

Campo	Tipo	Observación
Cantidad / PrecioUnitario / Subtotal	int / decimal / calculado	Detalle de línea

Entregas

Campo	Tipo	Observación
IdEntrega	int (PK, identity)	Autonumérico
IdPedido	int (FK → Pedidos)	Pedido entregado
DireccionEntrega	varchar(255)	Obligatorio
EstadoEntrega	varchar(50)	Ej. en camino, entregado
FechaEstimada / FechaEntrega	date	Opcionales
UbicacionReferencia	varchar(255)	Opcional

Facturas

Campo	Tipo	Observación
IdFactura	int (PK, identity)	Autonumérico
IdPedido	int (FK → Pedidos)	Pedido facturado
NumeroFactura	varchar(50)	Único
FechaFactura	datetime	Opcional
Subtotal / Descuento / Total	decimal(10,2)	Montos

Calificaciones

Campo	Tipo	Observación
IdCalificacion	int (PK, identity)	Autonumérico
IdPedido	int (FK → Pedidos)	Pedido calificado
Estrellas	int	Escala de calificación
Comentario	varchar(500)	Opcional
FechaCalificacion	datetime	Por defecto GETDATE()

Cupones

Campo	Tipo	Observación
IdCupon	int (PK, identity)	Autonumérico
Codigo	varchar(50)	Único
Descripcion	varchar(255)	Opcional
PorcentajeDescuento / MontoDescuento	decimal(5,2) / decimal(10,2)	Uno u otro tipo de descuento
FechaInicio / FechaFin	date	Vigencia
Activo	bit	Habilitado / deshabilitado

MetodosPago

Campo	Tipo	Observación
IdMetodoPago	int (PK, identity)	Autonumérico
NombreMetodo	varchar(50)	Ej. tarjeta, efectivo, SINPE

Personalizaciones

Campo	Tipo	Observación
IdPersonalizacion	int (PK, identity)	Autonumérico
IdUsuario	int (FK → Usuarios)	Cliente que personaliza
IdCaja	int (FK → MysticBox)	Caja base
TamanoCaja	varchar(50)	Ej. pequeña, mediana, grande
Preferencias / Exclusiones	varchar(300)	Texto libre
MensajePersonalizado	varchar(255)	Opcional
FechaPersonalizacion	datetime	Opcional

WhiteList

Campo	Tipo	Observación
IdWhiteList	int (PK, identity)	Autonumérico
Correo	varchar(100)	Único
Activo	bit	Habilitado / deshabilitado
FechaRegistro	datetime	Opcional

7. Backend — API REST

La API expone 17 controladores REST bajo el prefijo /api, todos documentados automáticamente mediante Swagger (disponible en el entorno de desarrollo en la ruta /swagger). La mayoría de los controladores sigue una convención CRUD estándar (GET, GET por id, POST, PUT, DELETE).

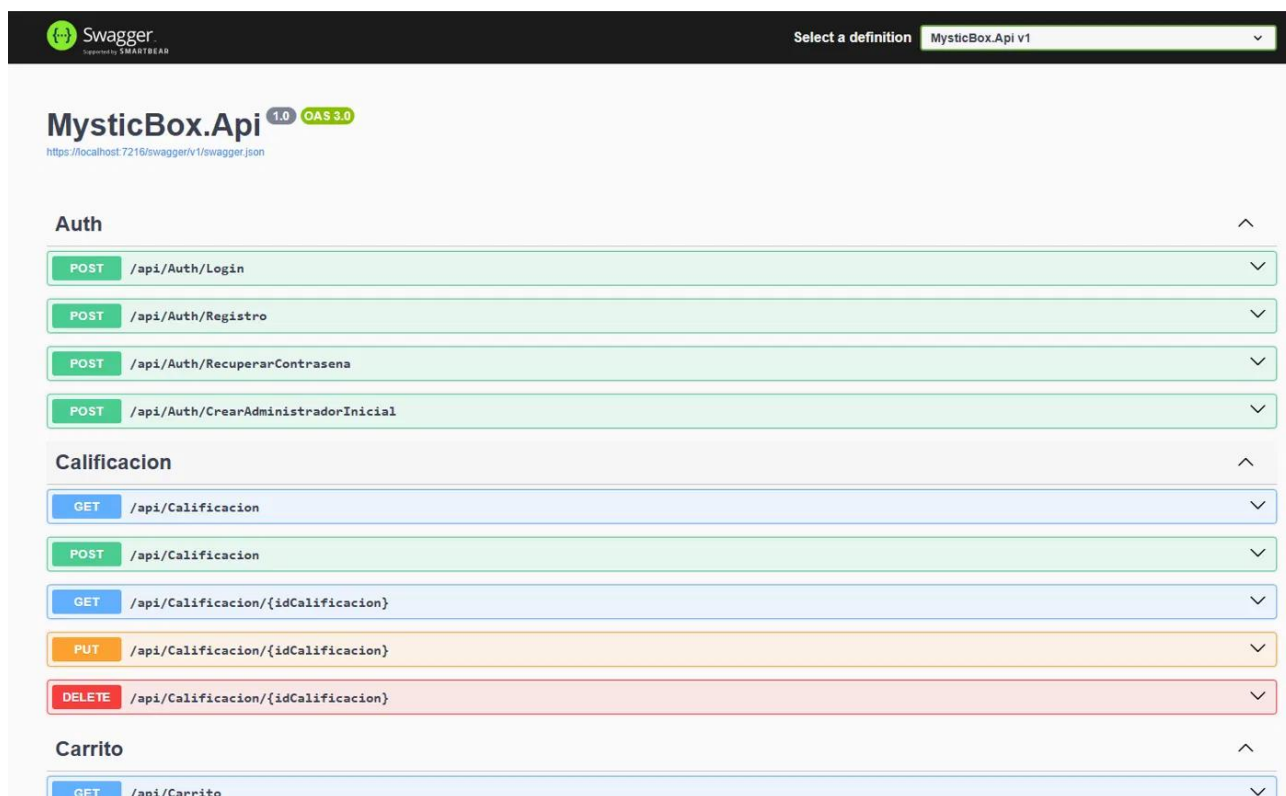


Figura 3. Swagger UI con los endpoints de la API REST de Mystic Box.

7.1 Catálogo de endpoints

AuthController (api/Auth)

Método	Ruta	Descripción
POST	/Login	Autenticación de usuario (correo + contraseña)
POST	/Registro	Registro de nuevo cliente
POST	/RecuperarContrasena	Recuperación de contraseña
POST	/CrearAdministradorInicial	Creación del administrador inicial del sistema

UsuarioController (api/Usuario)

Método	Ruta	Descripción
GET	/clientes	Lista de usuarios con rol cliente
GET	/id	Obtiene un usuario
PUT	/id	Actualiza un usuario

RolController (api/Rol)

Método	Ruta	Descripción
GET	/	Lista de roles
GET	/{{idRol}}	Obtiene un rol
POST	/	Crea un rol
PUT	/{{idRol}}	Actualiza un rol
DELETE	/{{idRol}}	Elimina un rol

CategoriasController (api/categorias)

Método	Ruta	Descripción
GET	/	Lista de categorías
GET	/{{id}}	Obtiene una categoría
POST	/	Crea una categoría
PUT	/{{id}}	Actualiza una categoría
DELETE	/{{id}}	Elimina una categoría

MysticBoxController (api/mysticbox)

Método	Ruta	Descripción
GET	/	Lista de cajas del catálogo
GET	/{{idCaja}}	Obtiene una caja
POST	/	Crea una caja
PUT	/{{idCaja}}	Actualiza una caja
DELETE	/{{idCaja}}	Elimina una caja

PersonalizacionesController (api/Personalizaciones)

Método	Ruta	Descripción
GET	/	Lista de personalizaciones
GET	/{{id}}	Obtiene una personalización
POST	/	Crea una personalización
PUT	/{{id}}	Actualiza una personalización

CarritoController (api/Carrito)

Método	Ruta	Descripción
GET	/	Lista de carritos
GET	/{{id}}	Obtiene un carrito
POST	/	Crea un carrito
PUT	/{{id}}	Actualiza un carrito
DELETE	/{{id}}	Elimina un carrito

DetalleCarritoController (api/DetalleCarrito)

Método	Ruta	Descripción
GET	/	Lista de líneas de carrito
GET	/id	Obtiene una línea
POST	/	Agrega una línea al carrito
PUT	/id	Actualiza una línea
DELETE	/id	Elimina una línea

CuponesController (api/cupones)

Método	Ruta	Descripción
GET	/	Lista de cupones
GET	/id	Obtiene un cupón
GET	/validar/codigo	Valida un código de cupón
POST	/	Crea un cupón
PUT	/id	Actualiza un cupón
DELETE	/id	Elimina un cupón

PedidoController (api/Pedido)

Método	Ruta	Descripción
GET	/	Lista de pedidos
GET	/id	Obtiene un pedido
GET	/(consulta adicional)	Consulta filtrada de pedidos
POST	/	Crea un pedido
PUT	/id	Actualiza un pedido
DELETE	/id	Elimina un pedido

DetallePedidoController (api/DetallePedido)

Método	Ruta	Descripción
GET	/	Lista de líneas de pedido
GET	/id	Obtiene una línea
POST	/	Crea una línea de pedido
PUT	/id	Actualiza una línea
DELETE	/id	Elimina una línea

EntregaController (api/Entrega)

Método	Ruta	Descripción
GET	/	Lista de entregas

Método	Ruta	Descripción
GET	/id	Obtiene una entrega
POST	/	Registra una entrega
PUT	/id	Actualiza el estado de una entrega
DELETE	/id	Elimina una entrega

FacturasController (api/facturas)

Método	Ruta	Descripción
GET	/	Lista de facturas
GET	/id	Obtiene una factura
POST	/	Genera una factura
PUT	/id	Actualiza una factura
DELETE	/id	Elimina una factura

CalificacionController (api/Calificacion)

Método	Ruta	Descripción
GET	/	Lista de calificaciones
GET	/id	Obtiene una calificación
POST	/	Registra una calificación
PUT	/id	Actualiza una calificación
DELETE	/id	Elimina una calificación

WhiteListController (api/whiteList)

Método	Ruta	Descripción
GET	/	Lista de correos autorizados
GET	/id	Obtiene un registro
POST	/	Agrega un correo a la lista blanca
PUT	/id	Actualiza un registro
DELETE	/id	Elimina un registro

DashboardController (api/Dashboard)

Método	Ruta	Descripción
GET	/	Indicadores generales para el panel administrativo

EstadisticasController (api/Estadisticas)

Método	Ruta	Descripción
GET	/	Estadísticas de ventas / satisfacción para reportes

7.2 Autenticación

El AuthController gestiona el ciclo de identidad del usuario: inicio de sesión (Login), registro de nuevos clientes (Registro), recuperación de contraseña y la creación del administrador inicial del sistema. La verificación de credenciales usa PasswordHasher<Usuario> de ASP.NET Core Identity, por lo que las contraseñas se almacenan como hash (nunca en texto plano) y se validan con VerifyHashedPassword.

Es importante señalar que el mecanismo de sesión actual no emite un token de acceso (JWT) ni protege los demás controladores con un esquema de autorización; el frontend simplemente conserva los datos del usuario autenticado en localStorage y decide qué rutas mostrar según el rol. Esta consideración se amplía en la sección de Seguridad.

8. Frontend — Aplicación Ionic + Angular

8.1 Estructura de rutas



Figura 4. Pantalla de inicio de sesión y recuperación de contraseña.

El enrutamiento se define en src/app/app.routes.ts usando componentes standalone cargados de forma perezosa (loadComponent). Cada ruta protegida declara sus guardas de acceso (canActivate). La tabla siguiente resume las rutas disponibles:

Ruta	Acceso	Descripción
/login	Pública	Inicio de sesión
/registro	Pública	Registro de nuevos clientes
/home	Cliente	Catálogo / inicio del cliente
/categorias	Cliente	Listado de categorías
/mysticbox	Cliente	Detalle de una caja
/personalizacion	Cliente	Personalización de una caja
/carrito	Cliente	Carrito de compras
/proceso-compra	Cliente	Checkout / proceso de compra
/factura	Cliente	Visualización de factura
/pedidos	Cliente	Historial de pedidos
/entregas	Cliente	Seguimiento de entregas
/calificaciones	Cliente	Calificación del pedido
/perfil	Autenticado	Perfil del usuario (cliente o admin)
/admin	Administrador	Panel principal de administración
/usuarios	Administrador	Gestión de usuarios
/roles	Administrador	Gestión de roles
/admin-cajas	Administrador	Gestión del catálogo de cajas
/admin-promociones	Administrador	Gestión de cupones/promociones
/admin-pedidos	Administrador	Gestión de pedidos
/admin-clientes	Administrador	Gestión de clientes
/admin-dashboard	Administrador	Panel de indicadores
/admin-reportes	Administrador	Reportes
/admin-reportes-generales	Administrador	Reportes generales
/admin-estadisticas	Administrador	Estadísticas de ventas
/admin-satisfaccion	Administrador	Estadísticas de satisfacción
/admin-whitelist	Administrador	Gestión de lista blanca de correos

8.2 Guardas de acceso (Guards)

Guard	Comportamiento
authGuard	Verifica que exista una sesión activa (AuthService.estaAutenticado()); si no la hay, redirige a /login.
adminGuard	Requiere sesión activa y rol Administrador (idRol === 1 o nombreRol "administrador"/"admin"); si el usuario no es administrador, redirige a /home.
clienteGuard	Restringe el acceso a rutas exclusivas del perfil Cliente.

8.3 Servicios (capa de acceso a la API)

Cada entidad del backend cuenta con un servicio Angular dedicado en `src/app/services/`, que encapsula las llamadas HTTP correspondientes usando `HttpClient`: `auth.service.ts`, `usuario.service.ts`, `categorias.service.ts`, `mysticbox.ts`, `personalizacion.ts`, `carrito.service.ts`, `cupon.service.ts`, `pedido.service.ts`, `detallePedido.service.ts`, `entrega.service.ts`, `calificacion.service.ts`, `whitelist.service.ts`, `factura.service.ts`, `dashboard.service.ts`, `estadisticas.service.ts` y `satisfaccion.service.ts`. Todos consumen la URL base definida en `environment.apiUrl`.

8.4 Manejo de sesión en el cliente



Figura 5. Pantalla principal de la aplicación para el perfil Cliente.

AuthService gestiona la sesión sin un token de servidor: al iniciar sesión, guarda el objeto `UsuarioSesion` recibido del backend en `localStorage` bajo la llave "usuario"; `estaAutenticado()` comprueba su existencia; `esAdministrador()/esCliente()` derivan el rol del usuario almacenado para decidir la navegación (`obtenerRutaInicial()`); `cerrarSesion()` limpia tanto `localStorage` como `sessionStorage` (incluyendo carrito, producto seleccionado y filtros de búsqueda).

8.5 Empaquetado como aplicación Android (APK)

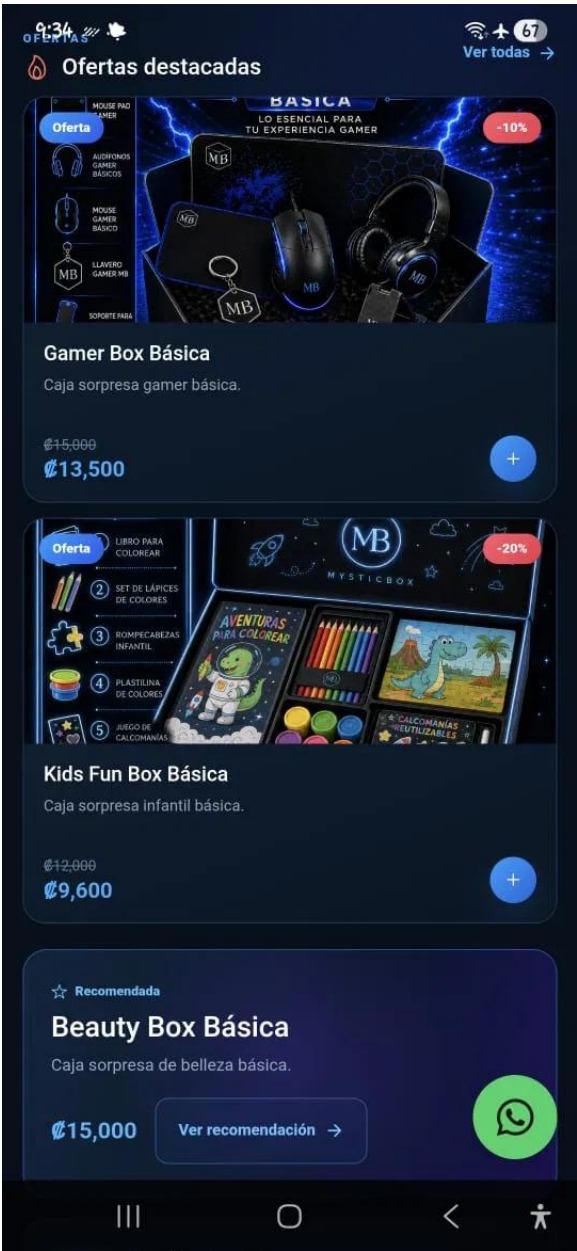


Figura 6. Aplicación Mystic Box ejecutándose en un dispositivo Android.

El proyecto incorpora la plataforma nativa de Android generada por Capacitor (carpeta MysticBoxApp/android), lo que permite compilar la aplicación Ionic/Angular como un archivo .apk instalable en un dispositivo o emulador Android, además de ejecutarse como aplicación web.

Para ello se realizaron los siguientes ajustes de configuración respecto a la versión web:

Archivo	Ajuste realizado
capacitor.config.ts	appName definido como "Mystic Box"; se agrega server.androidScheme: 'http' para que el WebView de Android cargue los recursos por HTTP.
android/app/src/main/AndroidManifest.xml	Se habilita android:usesCleartextTraffic="true" en la etiqueta <application>, necesario para que la app pueda consultar la API por HTTP plano en la red local (sin TLS).

Archivo	Ajuste realizado
src/environments/environment.ts y environment.prod.ts	apiUrl apunta a la IP LAN del equipo que ejecuta el backend (por ejemplo, http://192.168.18.153:5071/api) en lugar de https://localhost:7216/api, ya que el dispositivo Android no puede resolver "localhost" como el equipo de desarrollo.
Backend/MysticBox.Api/Properties/launchSettings.json	El backend se configura para escuchar en http://0.0.0.0:5071, exponiendo el puerto en toda la red local y no solo en localhost.
Backend/MysticBox.Api/Program.cs	La política de CORS se amplía temporalmente con .AllowAnyOrigin() y se comenta app.UseHttpsRedirection(), para permitir que la app empaquetada (que consume la API por HTTP) no sea bloqueada. Ver observación de seguridad en la sección 11.
package.json	Se agrega la dependencia @capacitor/android.

8.5.1 Pasos para generar el APK

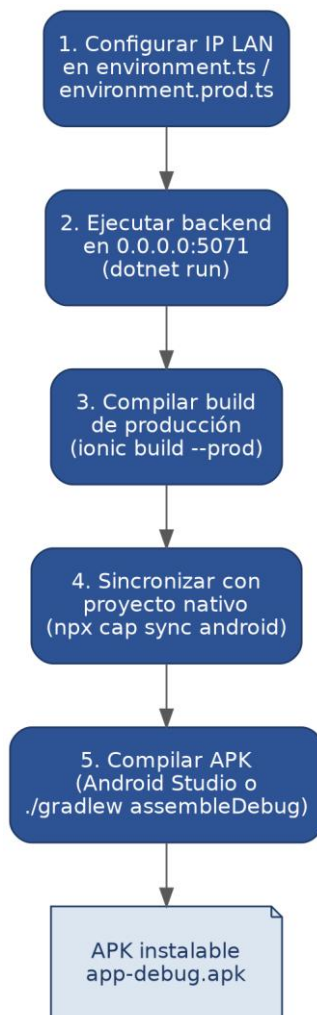


Figura 3. Flujo de generación del archivo APK.

1. Compilar la aplicación web de producción desde MysticBoxApp:

```
ionic build --prod
```

2. Sincronizar la salida (carpeta www) con el proyecto nativo de Android:

```
npx cap sync android
```

3. Abrir el proyecto en Android Studio para compilar el APK gráficamente:

```
npx cap open android
```

y desde Android Studio: Build → Build Bundle(s) / APK(s) → Build APK(s).

Alternativamente, compilar desde línea de comandos usando Gradle (dentro de la carpeta android):

```
cd android && ./gradlew assembleDebug
```

El APK de depuración resultante se genera en android/app/build/outputs/apk/debug/app-debug.apk. Para una versión de release firmada, se usa ./gradlew assembleRelease, lo cual requiere configurar previamente un keystore de firma.

9. Módulos Funcionales del Sistema

9.1 Gestión de usuarios y autenticación

Permite el registro de clientes, inicio de sesión, recuperación de contraseña y administración de usuarios y roles desde el panel administrativo (módulos Usuarios y Roles).

9.2 Catálogo y categorías

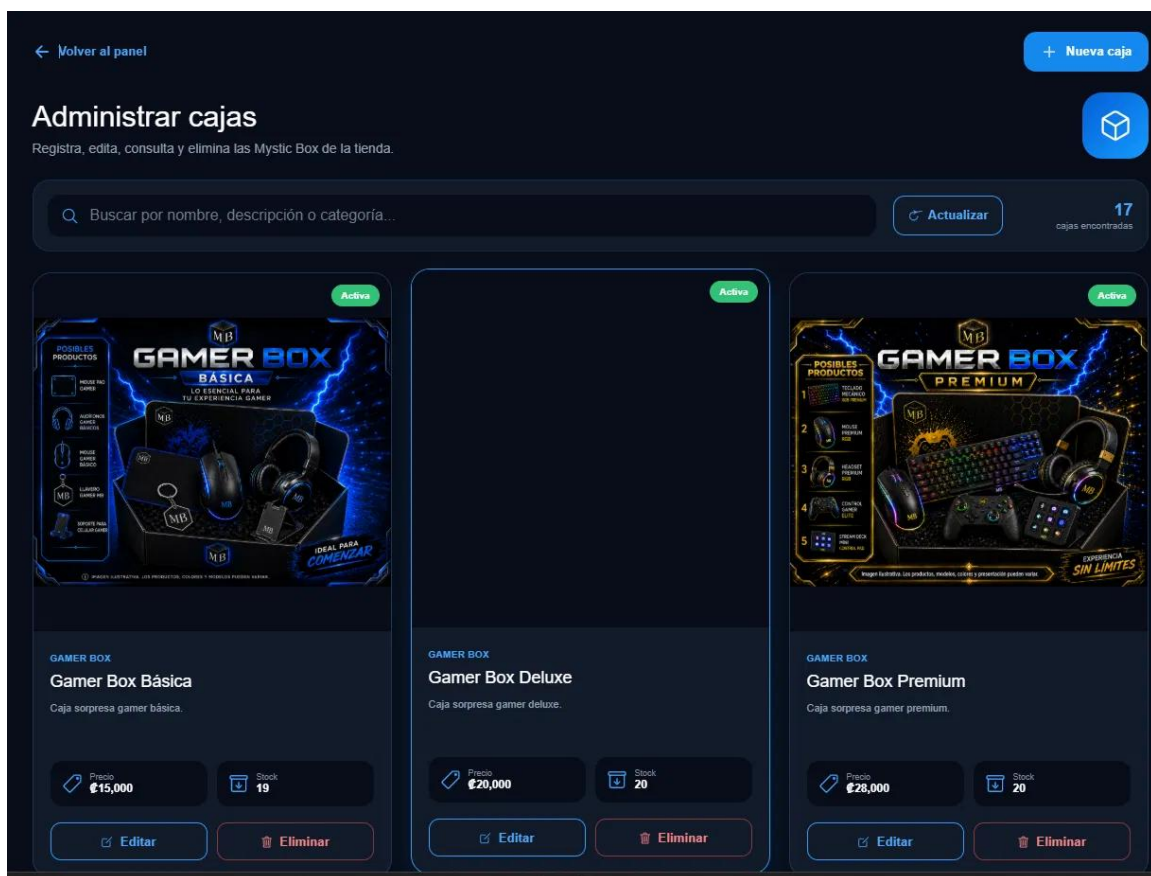


Figura 7. Módulo administrativo para consultar y gestionar las cajas del catálogo.

Administra las categorías de producto (Anime, Belleza, Gaming, Oficina, Autocuidado, Infantil) y el catálogo de cajas MysticBox asociadas a cada categoría, incluyendo precio, stock, imagen y estado (módulo Admin-Cajas).

9.3 Personalización de cajas

Permite al cliente definir el tamaño de caja, preferencias, exclusiones y un mensaje personalizado antes de agregarla al carrito o al pedido.

9.4 Carrito de compras

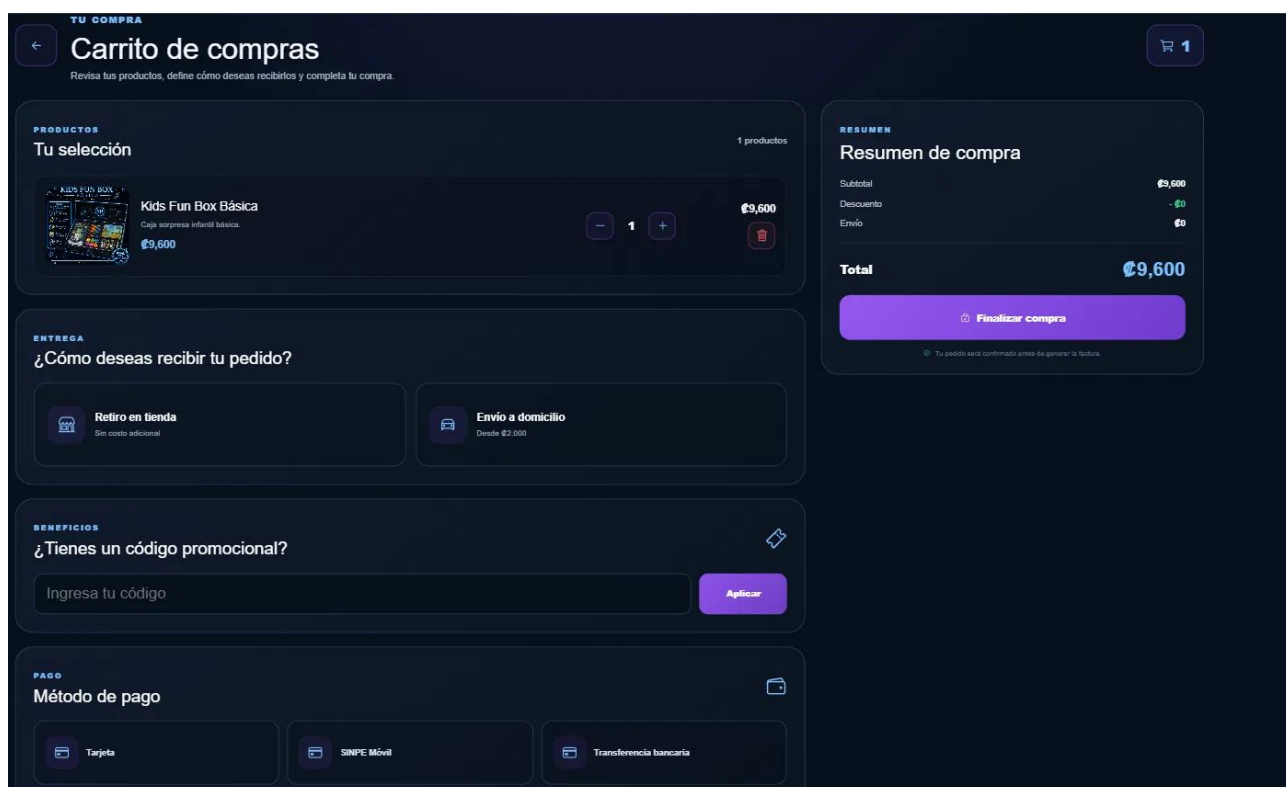


Figura 8. Carrito de compras con selección de entrega, cupón y método de pago.

Gestiona las líneas de producto seleccionadas por el cliente (cantidad, precio unitario y subtotal calculado automáticamente por la base de datos), previo a la generación del pedido.

9.5 Cupones y promociones

Administra códigos de descuento con vigencia (fecha de inicio/fin), porcentaje o monto fijo de descuento, y validación de código desde el proceso de compra (módulo Admin-Promociones).

9.6 Pedidos, facturación y métodos de pago

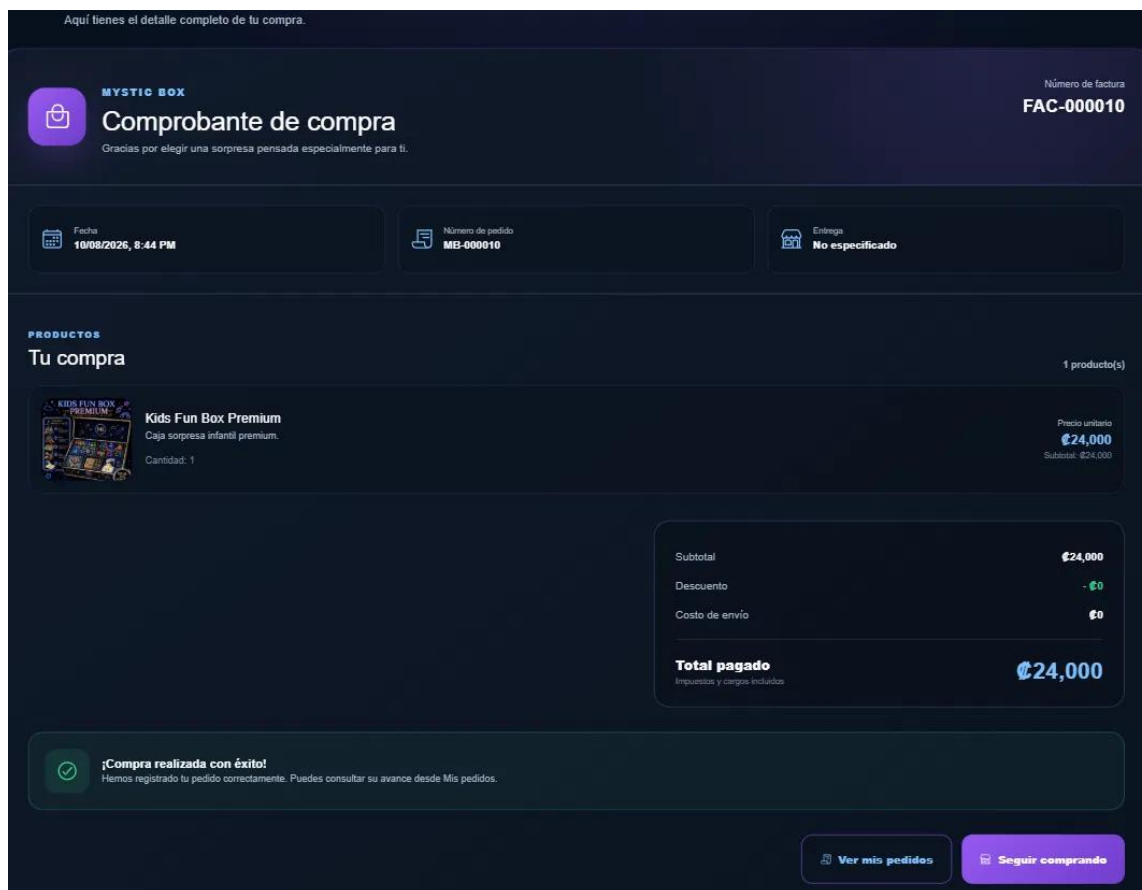


Figura 9. Comprobante de compra generado al finalizar un pedido.

Convierte el contenido del carrito en un pedido formal, aplicando el cupón (si corresponde) y el método de pago elegido; genera la factura asociada con número único, subtotal, descuento y total.

9.7 Entregas

Registra la dirección de entrega, el estado (pendiente, en camino, entregado), la fecha estimada y la fecha real de entrega de cada pedido.

9.8 Calificaciones

Permite al cliente calificar su experiencia (estrellas y comentario) una vez recibido el pedido, lo cual alimenta el módulo de satisfacción del panel administrativo.

9.9 Lista blanca (WhiteList)

Mantiene un listado de correos autorizados que el administrador puede gestionar, utilizado como control adicional de acceso o registro.

9.10 Panel administrativo: dashboard, reportes y estadísticas

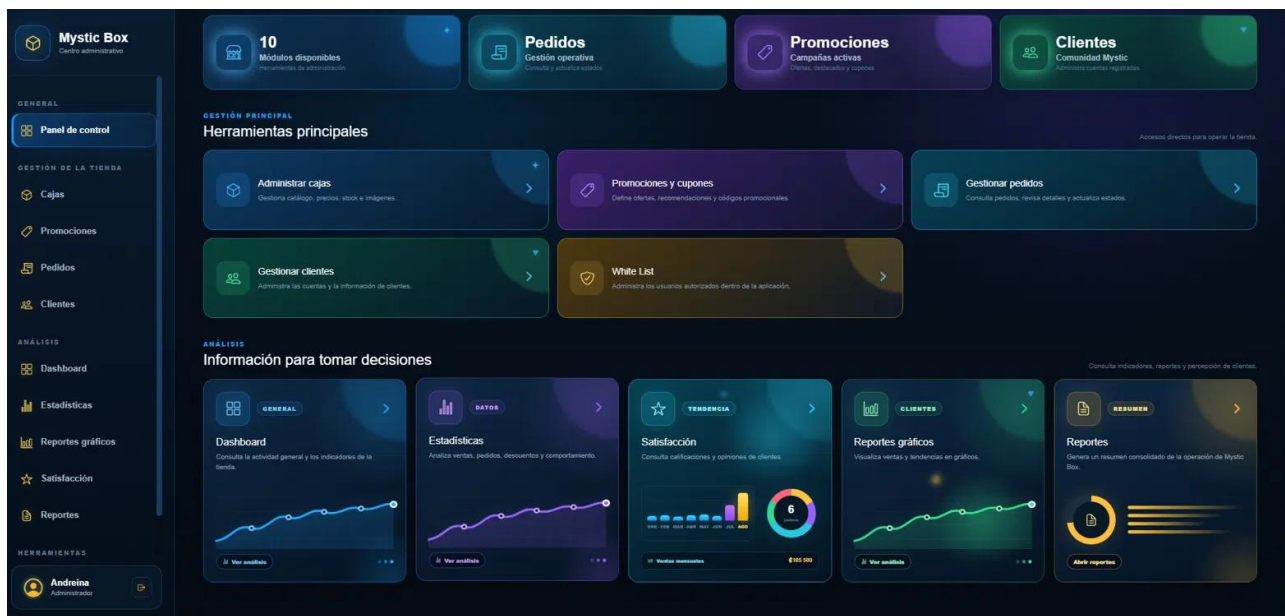


Figura 10. Panel administrativo con accesos a gestión, estadísticas y reportes.

Ofrece indicadores generales del negocio (DashboardController), reportes de ventas y operación (Admin-Reportes, Admin-Reportes-Generales) y estadísticas de ventas y satisfacción del cliente (EstadisticasController, Admin-Estadisticas, Admin-Satisfaccion).

10. Instalación y Configuración del Entorno

10.1 Requisitos previos

- .NET SDK 8.0 o superior
- SQL Server (Express o superior) con SQL Server Management Studio
- Node.js (versión compatible con Angular 20, recomendado Node 20 LTS o superior) y npm
- Ionic CLI (npm install -g @ionic/cli)
- Visual Studio 2022 (o VS Code) para el backend; VS Code recomendado para el frontend
- Git, para clonar y gestionar el repositorio del proyecto

10.2 Configuración de la base de datos

1. Crear una instancia de SQL Server accesible localmente (por ejemplo, SQLEXPRESS).
2. Ejecutar el script Database/BD_MysticBox.sql en SQL Server Management Studio para crear la base de datos BD_MysticBox, sus tablas, llaves primarias, foráneas y restricciones.
3. Verificar que exista un login/usuario de SQL Server con permisos sobre la base de datos, o utilizar autenticación integrada de Windows.

10.3 Configuración y ejecución del backend

1. Abrir la solución Backend/MysticBox.Api.slnx en Visual Studio (o el proyecto MysticBox.Api con dotnet CLI).
2. Editar la cadena de conexión en Backend/MysticBox.Api/appsettings.json (clave ConnectionStrings:DefaultConnection) para apuntar al servidor, base de datos, usuario y contraseña correctos del entorno local. Se recomienda no dejar credenciales reales en el archivo versionado; ver sección de Seguridad.

```
"DefaultConnection": "Server=<SERVIDOR>\\SQLEXPRESS;Database=BD_MysticBox;User
Id=<usuario>;Password=<contraseña>;TrustServerCertificate=True;"
```

3. Restaurar los paquetes NuGet (dotnet restore) y compilar la solución (dotnet build).
4. Ejecutar la API (dotnet run --project Backend/MysticBox.Api, o F5 en Visual Studio). Por defecto queda disponible en https://localhost:7216 (según launchSettings.json).
5. Verificar el funcionamiento accediendo a la interfaz de Swagger en https://localhost:7216/swagger.

10.4 Configuración y ejecución del frontend

1. Ubicarse en la carpeta MysticBoxApp y ejecutar npm install para instalar las dependencias declaradas en package.json.
2. Verificar que src/environments/environment.ts apunte a la URL correcta del backend (apiUrl), por defecto https://localhost:7216/api.
3. Levantar el servidor de desarrollo con ionic serve o npm start; la aplicación quedará disponible normalmente en http://localhost:8100.

4. Para compilar una versión de producción: `ionic build --prod` (o `ng build`). Para generar la aplicación móvil nativa: `npx cap add android` / `npx cap add ios` seguido de `npx cap sync`.

10.5 Generación del APK (aplicación Android)

El proyecto ya incluye la plataforma nativa Android (carpeta `MysticBoxApp/android`) configurada con Capacitor. Para generar el instalable `.apk`:

1. Determinar la dirección IP local (LAN) del equipo donde correrá el backend, por ejemplo con `ipconfig` (Windows) o `ifconfig/ip addr` (Linux/Mac).

2. Actualizar `src/environments/environment.ts` y `environment.prod.ts` en `MysticBoxApp` con esa IP, por ejemplo:

```
apiUrl: 'http://<IP_LAN_DEL_SERVIDOR>:5071/api'
```

3. Confirmar que el backend está configurado para escuchar en `0.0.0.0:5071` (ver `Backend/MysticBox.Api/Properties/launchSettings.json`) y ejecutarlo (`dotnet run`) para que quede accesible desde otros dispositivos de la misma red Wi-Fi.

4. Compilar la app web de producción y sincronizarla con el proyecto Android:

```
cd MysticBoxApp
ionic build --prod
npx cap sync android
```

5. Generar el APK, ya sea desde Android Studio (`npx cap open android` → Build → Build APK(s)) o por línea de comandos:

```
cd android && ./gradlew assembleDebug
```

El archivo resultante queda en `android/app/build/outputs/apk/debug/app-debug.apk`, listo para instalarse en un dispositivo Android (habilitando "orígenes desconocidos") o en un emulador. Ver la sección 8.5 para el detalle de los ajustes de configuración que habilitan este empaquetado.

10.6 Usuario administrador inicial

El endpoint `POST /api/Auth/CrearAdministradorInicial` permite crear el primer usuario con rol Administrador cuando la base de datos aún no cuenta con ninguno, habilitando así el acceso al panel administrativo por primera vez.

11. Seguridad y Consideraciones

11.1 Aspectos correctamente implementados

- Las contraseñas de usuario se almacenan como hash mediante `PasswordHasher<Usuario>` de ASP.NET Core Identity, nunca en texto plano.
- El backend valida credenciales vacías o inválidas antes de procesar el inicio de sesión.
- El acceso entre orígenes (CORS) está restringido explícitamente a los puertos de desarrollo de Ionic, en lugar de permitir cualquier origen.

del desarrollo.